

BUPT SE ICP Quiz -- Unit 09 File

If you are using LLM to help you in this quiz, you MUST write down 1) the name of the LLM and 2) the PROMPTS. Otherwise, you will get zero score if the instructor finds that you are using LLM.
You may use the following table as a template, putting the table down to the title of each question.

Name of LLM	Prompts
Qwen3-Max	I'm a beginner in learning ICP, and I'm now confused in File operation in C, plz interpret the following questions.

- [A] Which representation more closely corresponds to the internal representation of a text file: (a) a two-dimensional array consisting of a sequence of lines or (b) a one-dimension sequence of character?
- [F] True or false: The function getc returns a value of type char.
- [T] True or false: Every argument to scanf after the control string must be a pointer.
- [F] True or false: Every argument to scanf after the control string must be preceded with an ampersand (&).
- What is the purpose of the type FILE *? Is understanding the underlying structure of this type important to most programmers?

Name of LLM	Prompts
Qwen AI	I'm a beginner in learning ICP, plz interpret the following question in C (C99)

- FILE * is an **opaque pointer** to a structure that represents a stream (e.g., a file, stdin, stdout). It holds internal state such as buffer position, error flags, and file descriptor.
 - No**, most programmers **do not need to understand its internal structure**. The C standard I/O library (e.g., fopen, fread, fprintf) provides a high-level interface; you only need to use the FILE * as a handle, not inspect its contents.
- The second argument to fopen is usually one of the following string: "r", "w" or "a". What is the significance of this argument and what do each of these values mean?

The second argument is used to state what to do with that file.
"r" is used for READ ONLY; "w" for creating a new file and remove the former one if the file exists; "a" for append mode, open and write.
 - How does the fopen function report failure to its caller?

Return a "NULL" pointer.
 - The stdio.h interface automatically defines three standard files. What are their names? What purpose does each one serve?

"stdin", "stdout" and "stderr";
stdin — **standard input**: Typically reads from the keyboard (or redirected input).
 - stdout — **standard output**: Typically writes to the terminal (or redirected output).
 - stderr — **standard error**: Used for error messages; also goes to the terminal by default but can be redirected separately from stdout.

9. The formatted printing function comes in three forms: printf, fprintf, and sprintf. What are the differences?

Name of LLM	Prompts
Qwen AI	I'm a beginner in learning ICP, plz interpret the following question in C(C99)

- printf(const char *format, ...)
→ Writes formatted output to **stdout**.
- fprintf(FILE *stream, const char *format, ...)
→ Writes formatted output to a specified **file stream** (e.g., stdout, stderr, or a file opened with fopen).
- sprintf(char *str, const char *format, ...)
→ Writes formatted output into a **character buffer (string)** instead of a file or terminal.

10. What is the effect of the scanf specification "%10[^\n;]"?

scan all the input except "," and ";" and store these input into an array of 10;

11. Write a program to read words that input by the user and save the words into a text file "Words.txt". Try using both fputs and fprintf in your program.

```
#include <stdio.h>
#include <string.h>

#define MAX_WORD_LEN 100
#define FILENAME "Words.txt"

void saveWithFputs(const char *word, FILE *file);
void saveWithFprintf(const char *word, FILE *file);

int main() {
    FILE *file = fopen(FILENAME, "w");
    if (file == NULL) {
        printf("Error: Unable to create file %s\n", FILENAME);
        return 1;
    }

    char word[MAX_WORD_LEN];
    printf("Enter words (type 'STOP' to finish):\n");

    while (1) {
        printf("Word: ");
        if (scanf("%99s", word) != 1) {
            break; // Handle input error or EOF
        }

        if (strcmp(word, "STOP") == 0) {
```

```
        break;
    }

    saveWithFputs(word, file);
    saveWithFprintf(word, file);
}

fclose(file);
printf("Words saved to %s using both fputs and fprintf.\n",
FILENAME);
return 0;
}
```

12. Write a function `countWords()`, open “Words.txt” and count how many words are there in the file. Try using both `fgets` and `fscanf`. You also need to deal with the situation that the file does NOT exist.

```
#include <stdio.h>
#include <ctype.h>

typedef enum {
    COUNT_BY_FGETS,
    COUNT_BY_FSCANF
} CountMethod;

/*
 * Count words in "Words.txt".
 * A "word" is defined as a maximal sequence of non-whitespace
characters.
 *
 * Return value:
 *   ≥ 0 : number of words
 *   -1  : failed to open file (e.g., file does not exist)
 */
int countWords(CountMethod method) {
    const char *filename = "Words.txt";
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        perror("Failed to open Words.txt"); // includes "No such file
or directory" case
        return -1;
    }

    int count = 0;

    if (method == COUNT_BY_FSCANF) {
```

```
/* Method 1: fscanf reads tokens split by whitespace */
char word[1024];
while (fscanf(fp, "%1023s", word) == 1) {
    count++;
}
} else {
    /* Method 2: fgets reads line by line, then we scan characters
    */
    char line[4096];

    while (fgets(line, sizeof(line), fp) != NULL) {
        int in_word = 0; // 0: currently outside a word; 1: inside a
word
        for (int i = 0; line[i] != '\0'; i++) {
            unsigned char ch = (unsigned char)line[i];

            if (isspace(ch)) {
                in_word = 0;
            } else {
                if (!in_word) {
                    count++;
                    in_word = 1;
                }
            }
        }
    }
}

fclose(fp);
return count;
}

/* Example usage */
int main(void) {
    int n1 = countWords(COUNT_BY_FGETS);
    if (n1 >= 0) {
        printf("[fgets ] words = %d\n", n1);
    }

    int n2 = countWords(COUNT_BY_FSCANF);
    if (n2 >= 0) {
        printf("[fscanf] words = %d\n", n2);
    }

    return 0;
}
```

13. Based on Quiz12, write a function `search(char* keyword)` to search keyword in “Words.txt”. If find, print out the line that contains the keyword.

```
#include <stdio.h>
#include <string.h>

#define MAX_LINE 100
#define FILENAME "Words.txt"

void search(char* keyword) {
    FILE *file = fopen(FILENAME, "r");
    if (file == NULL) {
        printf("Error: File '%s' not found.\n", FILENAME);
        return;
    }

    char line[MAX_LINE];
    int found = 0;

    while (fgets(line, sizeof(line), file) != NULL) {
        // Remove newline if present
        line[strcspn(line, "\n")] = '\0';

        // Compare with keyword (exact match per line)
        if (strcmp(line, keyword) == 0) {
            printf("%s\n", line);
            found = 1;
        }
    }

    if (!found) {
        printf("Keyword '%s' not found in file.\n", keyword);
    }

    fclose(file);
}

// Optional: simple test main
int main() {
    char keyword[100];
    printf("Enter keyword to search: ");
    scanf("%99s", keyword);
    search(keyword);
    return 0;
}
```

14. Write a program `wc.c` that reads a file and report how many lines, words, and character appear in it. For the purposes of this program, a word consists of a consecutive sequence of any characters except white-space characters. For example, if the file `TaoTeChing.txt` contains the following passage from Lao Tsi,

The Tao that can be spoken or described is not the “Immutable and Everlasting Tao”.
The Name that can be named or spelt out is not the “immutable and everlasting Name”.
That which is “unidentifiable and nameless” is the Originator of the Universe,
whereas that which is “identifiable and with names” is the Mother of all things.

your program should be able to generate the following sample run:

```
File: TaoTeChing.txt8
Lines: 4
Words: 57
Chars: 325
```

```
#include <stdio.h>
```

```
int main() {
    char filename[100];
    printf("Enter filename: ");
    scanf("%99s", filename);

    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        printf("Error: Cannot open file '%s'\n", filename);
        return 1;
    }

    int lines = 0, words = 0, chars = 0;
    int in_word = 0;
    int ch;

    while ((ch = fgetc(file)) != EOF) {
        chars++;

        if (ch == '\n') {
            lines++;
        }

        if (ch == ' ' || ch == '\t' || ch == '\n') {
            in_word = 0;
        } else {
            if (!in_word) {
                words++;
                in_word = 1;
            }
        }
    }
}
```

```
}

if (chars > 0 && lines == 0) {
    lines = 1;
}

fclose(file);
printf("%d %d %d %s\n", lines, words, chars, filename);
return 0;
}
```

15. Write a program, Open a binary file “Numbers.dat” and write 10 double numbers to the file. Then read back these numbers and print them out.

```
#include <stdio.h>
int main() {
    const char* filename = "Numbers.dat";
    const int count = 10;
    double numbers[count] = {
        1.1, 2.2, 3.3, 4.4, 5.5,
        6.6, 7.7, 8.8, 9.9, 10.1
    };

    // Write 10 doubles to binary file
    FILE *file = fopen(filename, "wb");
    if (file == NULL) {
        printf("Error: Cannot open file for writing.\n");
        return 1;
    }

    fwrite(numbers, sizeof(double), count, file);
    fclose(file);
    printf("Wrote %d double numbers to '%s'.\n", count, filename);

    // Read them back
    double read_numbers[count];
    file = fopen(filename, "rb");
    if (file == NULL) {
        printf("Error: Cannot open file for reading.\n");
        return 1;
    }

    size_t read_count = fread(read_numbers, sizeof(double), count,
file);
    fclose(file);
```

```
// Print the numbers
printf("Numbers read from '%s':\n", filename);
for (size_t i = 0; i < read_count; i++) {
    printf("%.1f\n", read_numbers[i]);
}

return 0;
}
```

16. Write a program to read 2 students' information (including string name, int age and bool gender) input by the user and save the information into a file "Students.dat" in binary mode. You should construct a "Student" struct data type in your program and save each student's information just as a binary form struct.

```
#include <stdio.h>
#include <string.h>

#define NAME_LEN 50

typedef struct {
    char name[NAME_LEN];
    int age;
    int gender; // 1 for male, 0 for female
} Student;

//Dont know if this is correct [💧]
void inputStudent(Student* stu) {
    printf("Enter name: ");
    fgets((stu->name), NAME_LEN, stdin);
    printf("Enter age: ");
    scanf("%d", &stu->age);
    printf("Enter gender (1 = Male, 0 = Female): ");
    scanf("%d", &stu->gender);

    while (getchar() != '\n');
}

void saveFile(Student students[], int count) {
    FILE *file = fopen("Students.dat", "wb");
    if (file == NULL) {
        printf("Error: Cannot open file for writing.\n");
        return;
    }
    fwrite(students, sizeof(Student), count, file);
    fclose(file);
}
```



```
    printf("Student data saved to 'Students.dat' in binary format.\n");
}

int main() {
    Student students[2];

    for (int i = 0; i < 2; i++) {
        printf("--- Student %d ---\n", i + 1);
        inputStudent(&students[i]);
    }

    saveFile(students, 2);

    return 0;
}
```

17. Based on Quiz16, write a function insertRecord() which opens the file “Students.dat” and insert a new Student record after the first student record in the file.

```
#include <stdio.h>
#include <string.h>

#define NAME_LEN 50

typedef struct {
    char name[NAME_LEN];
    int age;
    int gender;
} Student;

void insertRecord() {
    Student students[2];
    Student newStudent;
    Student output[3];

    FILE *file = fopen("Students.dat", "rb");
    if (!file) {
        printf("Error: 'Students.dat' not found or unreadable.\n");
        return;
    }
    fread(students, sizeof(Student), 2, file);
    fclose(file);

    printf("\n--- New Student (to insert after first) ---\n");
    printf("Enter name: ");
    fgets(newStudent.name, NAME_LEN, stdin);
    size_t len = strlen(newStudent.name);
```

```
    if (len > 0 && newStudent.name[len - 1] == '\n')
        newStudent.name[len - 1] = '\\0';

    printf("Enter age: ");
    scanf("%d", &newStudent.age);
    printf("Enter gender (1=Male, 0=Female): ");
    scanf("%d", &newStudent.gender);
    while (getchar() != '\\n'); // flush

    output[0] = students[0];
    output[1] = newStudent;
    output[2] = students[1];

    file = fopen("Students.dat", "wb");
    if (!file) {
        printf("Error: Cannot write to file.\\n");
        return;
    }
    fwrite(output, sizeof(Student), 3, file);
    fclose(file);

    printf("Insertion complete. Total records: 3\\n");
}

int main() {
    insertRecord();
    return 0;
}
```

18. Based on Quiz17, write a function `insertRecordSorted()` which opens the file “Students.dat” and insert a new Student record according to the sequence of the name. Hint: you need to read in all the students/student names from the file and then find which position the new record should be insert into.

```
#include <stdio.h>
#include <string.h>

#define NAME_LEN 50
#define MAX_STUDENTS 10 // safe upper bound

typedef struct {
    char name[NAME_LEN];
    int age;
    int gender;
} Student;

void insertRecordSorted() {
    Student students[MAX_STUDENTS];
```

```
Student newStudent;
Student output[MAX_STUDENTS + 1]; // +1 for new student
int count = 0;

FILE *file = fopen("Students.dat", "rb");
if (file == NULL) {
    printf("Warning: 'Students.dat' not found. Starting fresh.\n");
    count = 0;
} else {
    while (fread(&students[count], sizeof(Student), 1, file) == 1)
    {
        count++;
        if (count ≥ MAX_STUDENTS) break; // safety
    }
    fclose(file);
}

printf("Current number of students: %d\n", count);

// new student
printf("\n--- Enter new student (will be inserted in name-sorted
order) ---\n");
printf("Enter name: ");
fgets(newStudent.name, NAME_LEN, stdin);
printf("Enter age: ");
scanf("%d", &newStudent.age);
printf("Enter gender (1 = Male, 0 = Female): ");
scanf("%d", &newStudent.gender);
while (getchar() ≠ '\n'); // AI said this

// Find insertion index
int insertIndex = count;
for (int i = 0; i < count; i++) {
    if (strcmp(newStudent.name, students[i].name) < 0) {
        insertIndex = i;
        break;
    }
}

// Build new array
int newCount = count + 1;
int outIndex = 0;

for (int i = 0; i < newCount; i++) {
    if (i == insertIndex) {
        output[outIndex++] = newStudent;
    }
}
```

```
        if (i < count) {
            output[outIndex++] = students[i];
        }
    }

    // Write
    file = fopen("Students.dat", "wb");
    if (file == NULL) {
        printf("Error: Cannot write to 'Students.dat'.\n");
        return;
    }
    fwrite(output, sizeof(Student), newCount, file);
    fclose(file);

    printf("Student inserted at position %d (0-based). Total records:
%d\n",
        insertIndex, newCount);
}
```